

NATURAL LANGUAGE PROCESSING · COMPANION READER

# Neural Networks & Neural Language Models

From one tiny unit, to networks, to the language models behind modern AI

Session 5 (Dr. Chetana Gavankar) · Read alongside the lecture slides · Every slide example worked out in full

**How to use this companion.** The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into plain English. Every slide example is worked out in full, step by step. Colour-coded boxes guide you. The **blue** “intuition” box states the core idea in one breath. The **amber** “everyday picture” box ties it to real life before any math. The **green** “worked example” box solves a problem with real numbers and no skipped steps. The **red** “watch out” box flags the common mistake. The **purple** “key takeaway” box is the one line to remember. Read it alongside the slides, in order.

## 1 What this session covers

This session climbs a short ladder. We start with one tiny computing unit, the building block of every deep network. We give it a smooth on/off switch called an activation. We meet a famous puzzle, the XOR problem, that one unit cannot solve. We fix it by stacking units into layers. Then we point these networks at language: classifying text, and predicting the next word. That next-word model is the seed of today’s large language models. We close with prompts, prompt engineering, and transfer learning.

Here is the path, in order:

- **Units** — the neuron: a weighted sum, a bias, and an activation.
- **Activations** — sigmoid, tanh, and ReLU, and how they differ.
- **The XOR problem** — why one unit (a perceptron) is too weak.
- **Feedforward networks** — many units in layers (an MLP).
- **Applying networks to NLP** — sentiment, and learned embeddings.
- **Neural language models** — predict the next word from context.
- **Training** — the loss, and one downhill step (gradient descent).
- **Large and small language models** — and prompt engineering.
- **Transfer learning** — reuse a big pre-trained model on a new task.

A quick note on the brain. A neuron is loosely inspired by a brain cell. Signals come in (like dendrites) and a signal goes out (like an axon). But please hold this lightly. This is *not* a model

of your brain. It is just a small function of numbers. The biology is a hint, not the truth.

## 2 The neural unit: the atom of deep learning

Everything in this session is built from one small object. We call it a **unit** (or a **neuron**). A unit takes a few numbers in, mixes them, and gives one number out. That is the whole atom. Once you understand one unit, a giant network is just many of them wired together.

### The intuition

A unit is a tiny vote-counter. Each input gets a **weight**, which says how much that input matters. The unit adds up the weighted votes, adds one extra number called the **bias**, and then squashes the total into a clean answer.

### ★ Everyday picture

Think about deciding whether to go for a walk. You weigh a few signals. Sunshine matters a lot, so it gets a big weight. Wind annoys you a little, so it gets a small negative weight. You also have a personal mood, a baseline nudge: that is the bias. You add it all up. If the total feels high enough, you go.

Let the inputs be  $x_1, x_2, \dots, x_n$ . These are just numbers, the features of one example. Each input  $x_i$  has a matching weight  $w_i$ , a number that scales it. There is one shared bias  $b$ , a single number added at the end. The unit first forms a **weighted sum**, which we call  $z$ :

$$z = b + \sum_{i=1}^n w_i x_i. \quad (1)$$

The symbol  $\sum_{i=1}^n$  just means “add these up for  $i$  from 1 to  $n$ .” We can write the same thing more compactly. Collect the weights into a vector  $\mathbf{w}$  and the inputs into a vector  $\mathbf{x}$ . The **dot product**  $\mathbf{w} \cdot \mathbf{x}$  multiplies matching parts and adds them. So:

$$z = \mathbf{w} \cdot \mathbf{x} + b. \quad (2)$$

The raw sum  $z$  can be any number, large or small, positive or negative. We want a clean, bounded answer. So we pass  $z$  through a **nonlinear activation**  $f$ , a curved function that bends the line. The output is  $y$ :

$$y = f(z) = f(\mathbf{w} \cdot \mathbf{x} + b). \quad (3)$$

The most common starter activation is the **sigmoid**, written  $\sigma$  (the Greek letter sigma). It squashes any number into the range between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (4)$$

Here  $e \approx 2.71828$  is Euler’s number, the base of natural logarithms. Putting it together, one sigmoid unit computes

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}. \quad (5)$$

### The neural unit: weigh, add a bias, then squash

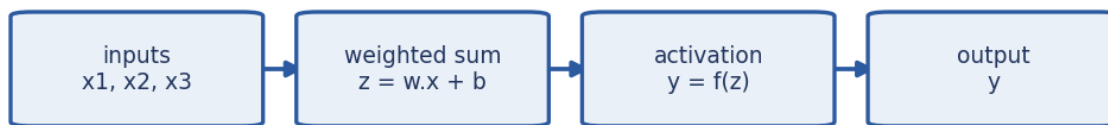


Figure 1: How to read it: numbers flow left to right. The unit weighs the inputs and adds a bias to get  $z$ , then squashes  $z$  through the activation  $f$  to get the output  $y$ .

This is exactly logistic regression, the classifier you may already know. So one unit is not a toy. It is a complete, working model on its own.

 **Worked example: the slide's neuron**,  $w = [0.2, 0.3, 0.9]$ ,  $b = 0.5$ ,  $x = [0.5, 0.6, 0.1]$

We have three inputs and three weights. The target is the output  $y$ .

**Step 1. Write down what we know.** Weights  $w = [0.2, 0.3, 0.9]$ , bias  $b = 0.5$ , inputs  $x = [0.5, 0.6, 0.1]$ .

**Step 2. Multiply each input by its weight.**  $0.2 \times 0.5 = 0.10$ ;  $0.3 \times 0.6 = 0.18$ ;  $0.9 \times 0.1 = 0.09$ .

**Step 3. Add the products (the dot product).**  $0.10 + 0.18 + 0.09 = 0.37$ .

**Step 4. Add the bias to get  $z$ .**  $z = 0.37 + 0.5 = \mathbf{0.87}$ .

**Step 5. Apply the sigmoid.**

$$y = \sigma(0.87) = \frac{1}{1 + e^{-0.87}}.$$

$$\text{Now } e^{-0.87} \approx 0.4190, \text{ so } y = \frac{1}{1 + 0.4190} = \frac{1}{1.4190} \approx \mathbf{0.7045}.$$

**Step 6. Sense-check.** Is it between 0 and 1? Yes. Is it above 0.5? Yes, so this unit leans toward “yes,” fairly confidently.

The answer  $y \approx 0.7045$  matches the slide. Every step is plain arithmetic.

**Where this shows up in ML.** A single sigmoid unit is logistic regression, used everywhere for yes/no decisions: spam or not, fraud or not. In a deep network, this same unit is repeated thousands of times. The whole network is layers of these atoms, each one a weighted sum followed by an activation.

#### Watch out

The bias is not optional. Without  $b$ , the unit can only separate inputs with a line through the origin. The bias is the knob that slides the decision anywhere. Also, the activation must be *nonlinear*. Without a bend, stacking units just gives one big line, and the network gains nothing.

**✓ Key takeaway**

One unit computes  $y = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$ : weigh the inputs, add a bias, squash the total. That single line is the building block of every neural network in this course.

### 3 Activation functions: the bend that gives power

The activation is the curved step inside a unit. Without it, a network is just one straight line, no matter how many layers it has. The bend is what lets networks learn rich, twisty patterns. We meet the three most common bends.

#### The intuition

An activation is a soft decision switch. It takes the raw score  $z$  and shapes it. Some switches squash into a fixed range. One switch simply turns negative scores off. The shape you pick changes how fast the unit learns.

#### ★ Everyday picture

Think of a dimmer on a light. A plain wire passes voltage straight through. A dimmer reshapes it: low stays low, high saturates near full brightness. The activation is that dimmer for a unit's score.

The **sigmoid**  $\sigma(z) = 1/(1 + e^{-z})$  squashes any number into the open range between 0 and 1. It is smooth and reads nicely as a probability. Its weakness: when  $z$  is very large or very small, the curve is almost flat, so learning slows.

The **tanh** (hyperbolic tangent) has the same S-shape but is centred on zero. Its range is between  $-1$  and  $1$ :

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \quad (6)$$

The **ReLU** (rectified linear unit) is the simplest and the most common today. It passes positive scores through unchanged and sets negative scores to zero:

$$\text{ReLU}(z) = \max(0, z). \quad (7)$$

ReLU has a sharp kink at zero, not a smooth curve. For positive  $z$ , its slope is a constant 1. That steady slope helps deep networks learn fast and avoids the flat-curve slowdown of the sigmoid.

#### Worked example: compare the three activations at $z = 2$ and $z = -2$

We feed the same two scores through each activation.

**Step 1. Sigmoid at  $z = 2$ .**  $\sigma(2) = 1/(1 + e^{-2}) = 1/(1 + 0.1353) \approx 0.881$ .

**Step 2. Sigmoid at  $z = -2$ .**  $\sigma(-2) = 1/(1 + e^2) = 1/(1 + 7.389) \approx 0.119$ .

**Step 3. tanh at  $z = 2$ .**  $\tanh(2) = (e^2 - e^{-2})/(e^2 + e^{-2}) = (7.389 - 0.135)/(7.389 + 0.135) \approx 0.964$ .

**Step 4. ReLU at  $z = 2$ .**  $\max(0, 2) = 2$ .

**Step 5. ReLU at  $z = -2$ .**  $\max(0, -2) = 0$ .

Notice the contrast. Sigmoid and tanh both saturate near their limits. ReLU just passes 2 through and zeroes out  $-2$ .

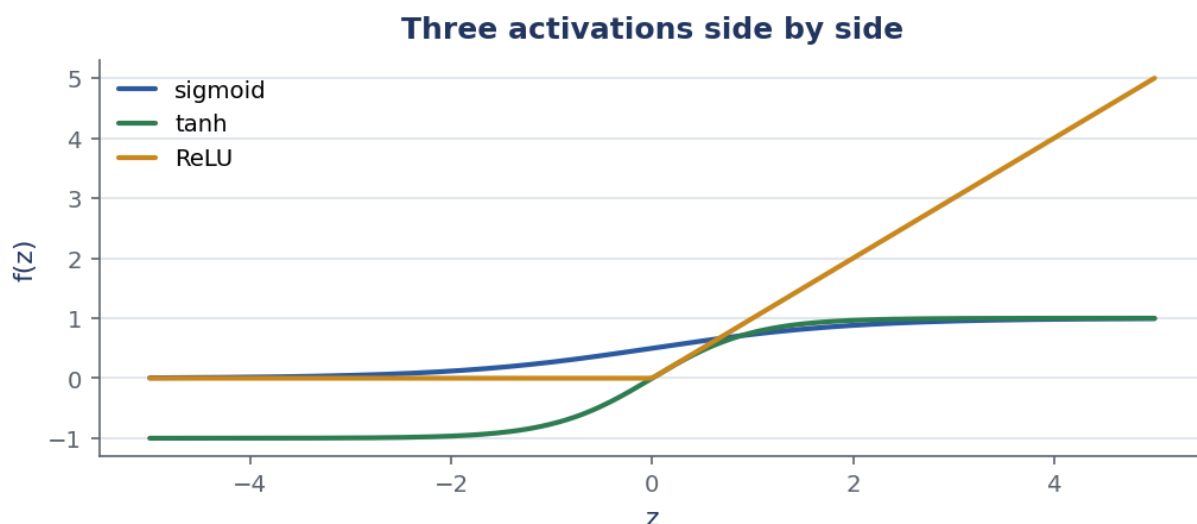


Figure 2: How to read it: all three activations on one axis. The sigmoid (0 to 1) and tanh (−1 to 1) both flatten at the edges. ReLU stays at zero on the left, then rises as a straight line.

**Where this shows up in ML.** ReLU is the default hidden activation in most deep networks, from image models to transformers. Sigmoid and tanh appear inside gates of recurrent networks, and sigmoid is the final layer for a yes/no output. The choice of activation is a real design decision, not a detail.

⚠ Watch out

Sigmoid and tanh “saturate”: far from zero, their slope is nearly flat, so gradients shrink and learning stalls. This is the vanishing-gradient trap. ReLU mostly avoids it, but a unit stuck at negative  $z$  can “die” and output zero forever. There is no perfect switch.

✓ Key takeaway

Sigmoid maps to  $(0, 1)$ , tanh to  $(-1, 1)$ , and ReLU keeps positives and zeroes negatives. ReLU is the common default because its constant positive slope helps deep networks train.

## 4 The perceptron, and AND/OR

Before networks, there was the **perceptron** (Rosenblatt, 1958). A perceptron is the simplest unit. It has weights and a bias, but no smooth activation. Its output is hard: 1 or 0. It fires a 1 when the weighted sum is above zero, and a 0 otherwise.

### The intuition

A perceptron is a strict gatekeeper. It adds up the weighted inputs plus the bias. If the total clears the bar (above zero), it says yes (1). If not, it says no (0). There is no “maybe” in between.

The rule, in symbols:

$$\text{output} = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0, \\ 0 & \text{otherwise.} \end{cases} \quad (8)$$

We can build basic logic gates with a perceptron. Take the **AND** gate. It should fire only when both inputs are 1. Use weights  $w = [1, 1]$  and bias  $b = -1$ . Then  $z = x_1 + x_2 - 1$ , and the gate fires when  $z > 0$ .

### Worked example: the AND perceptron, $w = [1, 1]$ , $b = -1$ , all four input rows

We test every pair  $(x_1, x_2)$  and read off the output.

**Step 1. Input** (0,0).  $z = 0 + 0 - 1 = -1$ . Not above 0, so output **0**.

**Step 2. Input** (0,1).  $z = 0 + 1 - 1 = 0$ . Not above 0, so output **0**.

**Step 3. Input** (1,0).  $z = 1 + 0 - 1 = 0$ . Not above 0, so output **0**.

**Step 4. Input** (1,1).  $z = 1 + 1 - 1 = 1$ . Above 0, so output **1**.

Only (1,1) fires. That is exactly the AND gate.

Now the **OR** gate. It should fire when at least one input is 1. Use weights  $w = [1, 1]$  and bias  $b = 0$ , with the rule “fire when  $z > 0$ .”

### Worked example: the OR perceptron, $w = [1, 1]$ , $b = 0$ , all four input rows

Again we test every pair and read the output.

**Step 1. Input** (0,0).  $z = 0 + 0 = 0$ . Not above 0, so output **0**.

**Step 2. Input** (0,1).  $z = 0 + 1 = 1$ . Above 0, so output **1**.

**Step 3. Input** (1,0).  $z = 1 + 0 = 1$ . Above 0, so output **1**.

**Step 4. Input** (1,1).  $z = 1 + 1 = 2$ . Above 0, so output **1**.

Every row except (0,0) fires. That is exactly the OR gate.

**Where this shows up in ML.** The perceptron is the historical seed of neural networks. Its update rule was the first learning algorithm for a linear classifier. Modern units keep its weighted-sum core. But they swap the hard 0/1 step for a smooth activation, so gradient descent can train them.



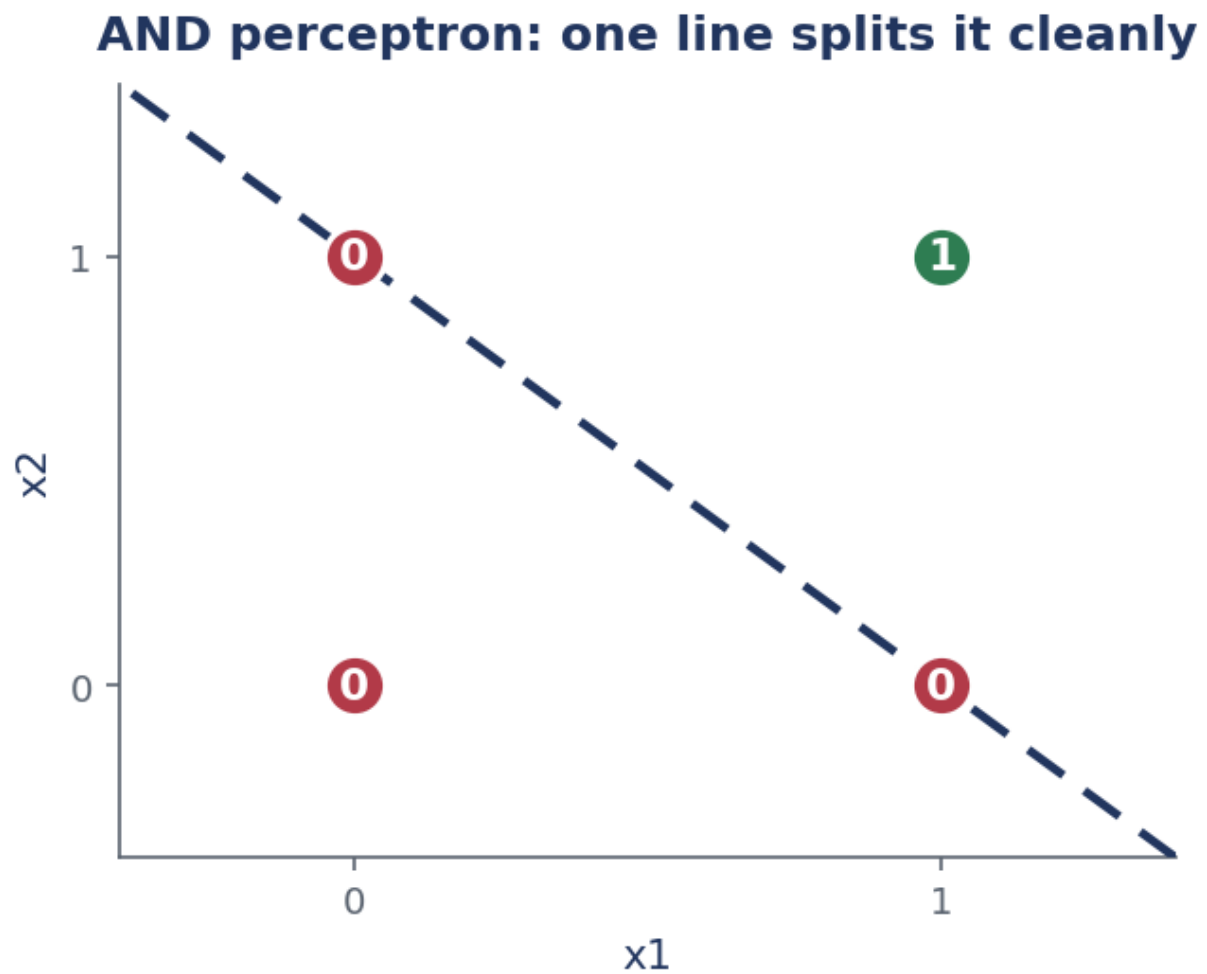


Figure 3: How to read it: the four corners are the inputs. Green is output 1, red is output 0. For AND, one dashed straight line cleanly puts the single green corner on one side.

**⚠ Watch out**

A perceptron draws one straight cut through the input space. The line  $\mathbf{w} \cdot \mathbf{x} + b = 0$  is its only weapon. It can solve AND and OR because each needs just one cut. It will fail the moment a task needs two cuts. We meet that task next.

**✓ Key takeaway**

A perceptron fires 1 when  $\mathbf{w} \cdot \mathbf{x} + b > 0$ , else 0. It is a linear classifier: one straight line. AND and OR fit on one line, so a single perceptron handles them.

## 5 The XOR problem: why one unit is not enough

In 1969, Minsky and Papert pointed out a sharp limit. One perceptron cannot compute **XOR** (exclusive or). XOR should fire when the two inputs *differ*: one is 1 and the other is 0. This simple gate broke the early perceptron dream and cooled the field for years.

### The intuition

XOR wants “fire when they disagree.” Look at the truth table below. The two “yes” cases,  $(0, 1)$  and  $(1, 0)$ , sit on opposite corners. So do the two “no” cases. You cannot fence the yeses off from the noes with one straight line. You would need a bent boundary, or two lines.

### ★ Everyday picture

Imagine two light switches at the top and bottom of a staircase. The light is on only when the switches disagree. Flip either one and the light flips. No single “threshold on the total” captures this. The total is the same (one switch on) for the two “on” cases and for neither of the “off” cases that share it.

The XOR truth table:

| $x_1$ | $x_2$ | XOR |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 1   |
| 1     | 0     | 1   |
| 1     | 1     | 0   |

Why can no single line work? A perceptron’s boundary is the line  $w_1x_1 + w_2x_2 + b = 0$ . That line splits the plane into two sides. XOR needs the green corners  $(0, 1)$  and  $(1, 0)$  on one side, and the red corners  $(0, 0)$  and  $(1, 1)$  on the other. But the green corners and red corners alternate around the square. They are **not linearly separable**: no straight line can keep the two greens together and away from the two reds.

### Worked example: show one line always fails on XOR

Suppose a line scored  $(0, 1)$  and  $(1, 0)$  as “yes” ( $z > 0$ ). We check if the same line can score both  $(0, 0)$  and  $(1, 1)$  as “no.”

**Step 1. Want  $(0, 1)$  positive.**  $z = w_2 + b > 0$ .

**Step 2. Want  $(1, 0)$  positive.**  $z = w_1 + b > 0$ .

**Step 3. Add those two.**  $(w_1 + b) + (w_2 + b) > 0$ , so  $w_1 + w_2 + 2b > 0$ .

**Step 4. Want  $(1, 1)$  negative.**  $z = w_1 + w_2 + b < 0$ .

**Step 5. Want  $(0, 0)$  negative.**  $z = b < 0$ .

**Step 6. Combine the last two.** From step 4,  $w_1 + w_2 + b < 0$ . Add  $b < 0$  to get  $w_1 + w_2 + 2b < 0$ . This clashes with step 3, which needed  $w_1 + w_2 + 2b > 0$ . A contradiction.

No weights and bias satisfy all four rows at once. So one perceptron can never do XOR.

**Where this shows up in ML.** XOR is the textbook example of a problem that needs depth. It shows why a single linear layer is not enough for real data, which is full of such “it depends”

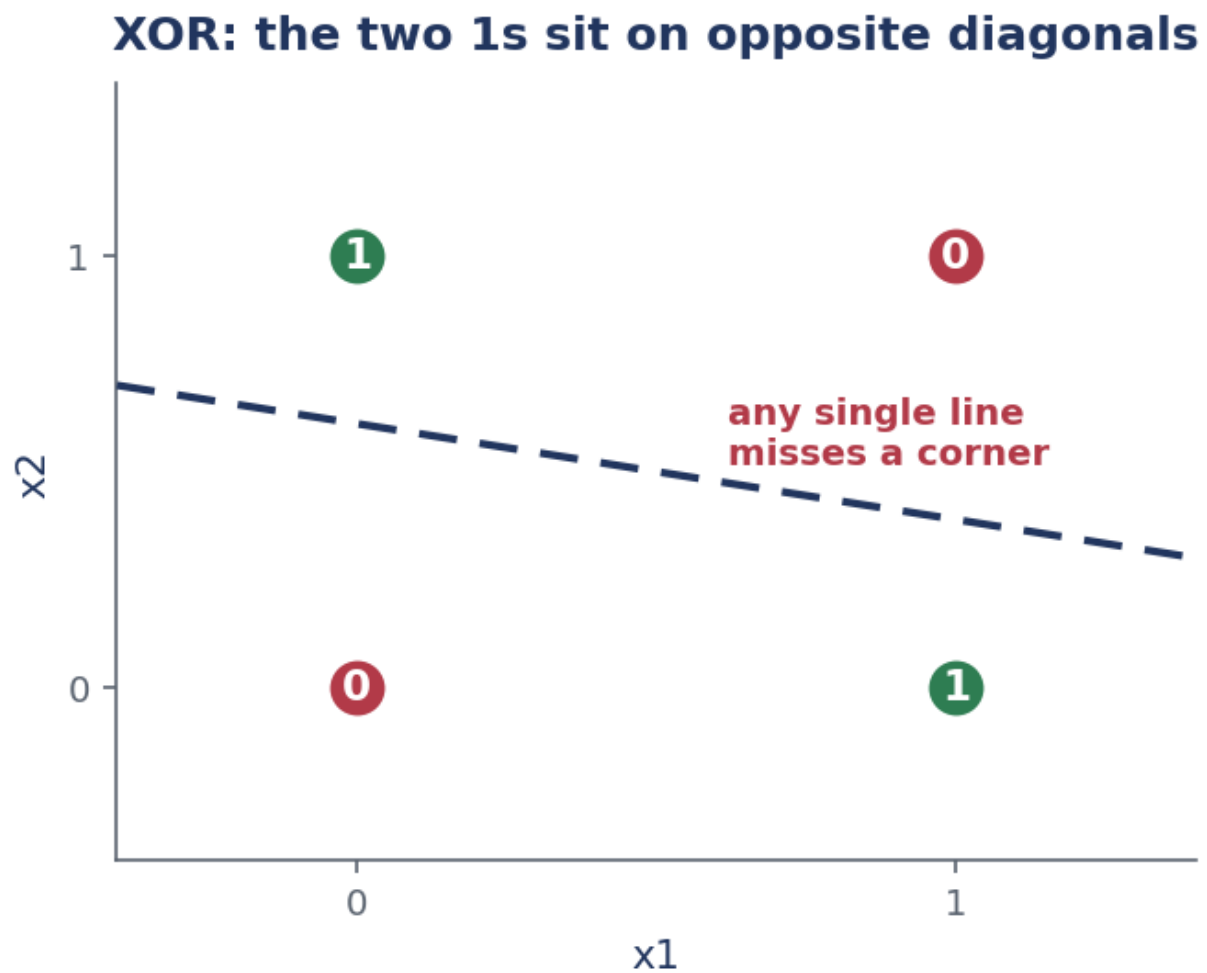


Figure 4: How to read it: green corners are output 1, red are output 0. The greens sit on one diagonal, the reds on the other. Any single straight line leaves at least one corner on the wrong side.

patterns. The fix, adding a hidden layer, is the reason deep networks exist.

**✗ Watch out**

Do not read “XOR is hard” as “XOR is complex.” XOR is tiny. The lesson is narrower: one linear cut is too weak. The instant a task needs a bent or split boundary, a single unit fails, no matter how you tune its weights.

**✓ Key takeaway**

XOR fires when inputs differ. Its yes-corners and no-corners alternate, so they are not linearly separable. One perceptron, which draws one line, cannot solve it. We need layers.

## 6 Solving XOR with a 2-layer network

The fix is to stack units into layers. We add a **hidden layer** between the input and the output. The hidden layer first transforms the inputs into a new, more useful set of numbers. In that new space, the problem becomes linearly separable, and the output unit can finish the job.

### The intuition

The hidden layer is a translator. It re-describes the raw inputs as features that make the answer easy. For XOR, one hidden unit learns “at least one input is on,” and the other learns “both inputs are on.” From those two facts, XOR is just “at least one on, but not both.”

### ★ Everyday picture

Suppose you cannot sort a pile of coins by value directly. First you sort them by size, then by colour. Two simple sorts, done in sequence, solve a task neither could alone. The hidden layer is that first sort: it reshapes the data so the final step is easy.

Here is the standard 2-layer ReLU network from Jurafsky and Martin. The hidden layer uses a weight matrix and a bias vector:

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{h} = \text{ReLU}(W\mathbf{x} + \mathbf{b}). \quad (9)$$

The output unit takes the hidden vector  $\mathbf{h} = [h_1, h_2]$  and combines it with output weights and no bias:

$$\mathbf{w}_{\text{out}} = [1, -2], \quad b_{\text{out}} = 0, \quad y = \mathbf{w}_{\text{out}} \cdot \mathbf{h}. \quad (10)$$

Each hidden unit does a familiar job. Unit  $h_1$  computes  $\text{ReLU}(x_1 + x_2 + 0)$ : it counts how many inputs are on. Unit  $h_2$  computes  $\text{ReLU}(x_1 + x_2 - 1)$ : it only switches on when *both* inputs are on. Let us verify all four inputs.

### A 2-layer ReLU network that solves XOR

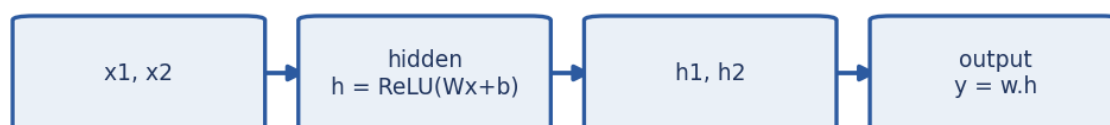


Figure 5: How to read it: inputs feed a hidden ReLU layer that produces  $h_1, h_2$ . The output unit mixes those with weights  $[1, -2]$  to give  $y$ . The hidden layer is what makes XOR solvable.

### Worked example: verify the 2-layer XOR net on all four inputs

For each input we compute  $h_1, h_2$ , then  $y = h_1 - 2h_2$ .

**Step 1.** Input  $x = [0, 0]$ .  $h_1 = \text{ReLU}(0+0+0) = 0$ ;  $h_2 = \text{ReLU}(0+0-1) = \text{ReLU}(-1) = 0$ . Then  $y = 0 - 2(0) = 0$ . Correct (XOR of 0,0 is 0).

**Step 2.** Input  $x = [0, 1]$ .  $h_1 = \text{ReLU}(0+1+0) = 1$ ;  $h_2 = \text{ReLU}(0+1-1) = \text{ReLU}(0) = 0$ . Then  $y = 1 - 2(0) = 1$ . Correct.

**Step 3.** Input  $x = [1, 0]$ .  $h_1 = \text{ReLU}(1+0+0) = 1$ ;  $h_2 = \text{ReLU}(1+0-1) = \text{ReLU}(0) = 0$ . Then  $y = 1 - 2(0) = 1$ . Correct.

**Step 4.** Input  $x = [1, 1]$ .  $h_1 = \text{ReLU}(1+1+0) = 2$ ;  $h_2 = \text{ReLU}(1+1-1) = \text{ReLU}(1) = 1$ . Then  $y = 2 - 2(1) = 0$ . Correct.

All four outputs match XOR: 0, 1, 1, 0. The hidden unit  $h_2$  is the key. It fires only on (1, 1), and the output subtracts twice its value to cancel that case.

**Where this shows up in ML.** This tiny example is the whole reason for “deep” learning. Hidden layers learn useful intermediate features, called a **representation**. A vision network’s early layers find edges, then shapes, then objects. Each layer reshapes the data so the next layer’s job is simpler.

**✗ Watch out**

The magic is not the output weights; it is the hidden layer’s new features. Remove the hidden layer and keep only a linear output. Then you are back to one line, and XOR fails again. Depth plus a nonlinear activation is what buys the power.

**✓ Key takeaway**

A hidden ReLU layer rewrites the inputs as features (here, “any on” and “both on”). In that new space XOR is linearly separable, so a 2-layer network solves it. Depth creates useful representations.

## 7 Feedforward networks and the softmax

A **feedforward neural network** is just many units, arranged in layers, with information flowing one way: input to output, no loops. Another name for it is a **multi-layer perceptron** (MLP). We now set up the notation we will use for the rest of the session, and meet the softmax for many-class outputs.

### The intuition

A feedforward network is an assembly line. Each layer takes the previous layer's numbers, applies its own weights and activation, and passes the result forward. The data never flows backward during prediction. It just marches from input to answer.

### 7.1 Logistic regression is a one-layer network

We do not count the input layer, because it does no computing; it just holds the numbers. So logistic regression, one sigmoid unit, is a **one-layer network**. It has a weight vector, a bias, and a single  $\sigma$  output. For several outputs at once, a fully-connected layer uses a weight *matrix*  $W$ , a bias *vector*  $\mathbf{b}$ , and produces a vector of scores.

### 7.2 Softmax: a probability for each of many classes

The sigmoid gives one probability, good for a yes/no choice. For  $k$  classes we need  $k$  probabilities that add to 1. The **softmax** does this. Given a score vector  $\mathbf{z} = [z_1, \dots, z_k]$ , the probability of class  $i$  is:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}. \quad (11)$$

Read it in two moves. First, make every score positive by raising  $e$  to it. Second, divide by the total of all of them, so the parts sum to one. A big score gets a big share. The softmax is the many-class big sister of the sigmoid.

### Worked example: the slide's softmax, $\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$

We turn six raw scores into six probabilities.

**Step 1. Raise  $e$  to each score.**  $e^{0.6} = 1.822$ ;  $e^{1.1} = 3.004$ ;  $e^{-1.5} = 0.223$ ;  $e^{1.2} = 3.320$ ;  $e^{3.2} = 24.533$ ;  $e^{-1.1} = 0.333$ .

**Step 2. Add them up (the bottom of the fraction).**  $1.822 + 3.004 + 0.223 + 3.320 + 24.533 + 0.333 = 33.235$ .

**Step 3. Divide each by the total.**  $1.822/33.235 = \mathbf{0.055}$ ;  $3.004/33.235 = \mathbf{0.090}$ ;  $0.223/33.235 = \mathbf{0.0067}$ ;  $3.320/33.235 = \mathbf{0.100}$ ;  $24.533/33.235 = \mathbf{0.738}$ ;  $0.333/33.235 = \mathbf{0.010}$ .

**Step 4. Sum-check.**  $0.055 + 0.090 + 0.0067 + 0.100 + 0.738 + 0.010 \approx \mathbf{1.000}$ . Good, they add to one.

The fifth class wins big, with about 74% of the mass, because its score 3.2 was far the largest. Softmax sharpens a lead into a clear winner.

### 7.3 Two layers, and the layer notation

A **two-layer network** adds one hidden layer. The hidden layer computes  $\mathbf{h} = g(W\mathbf{x} + \mathbf{b})$ , where  $g$  is a hidden activation like ReLU or tanh. The output is then a sigmoid (for yes/no) or



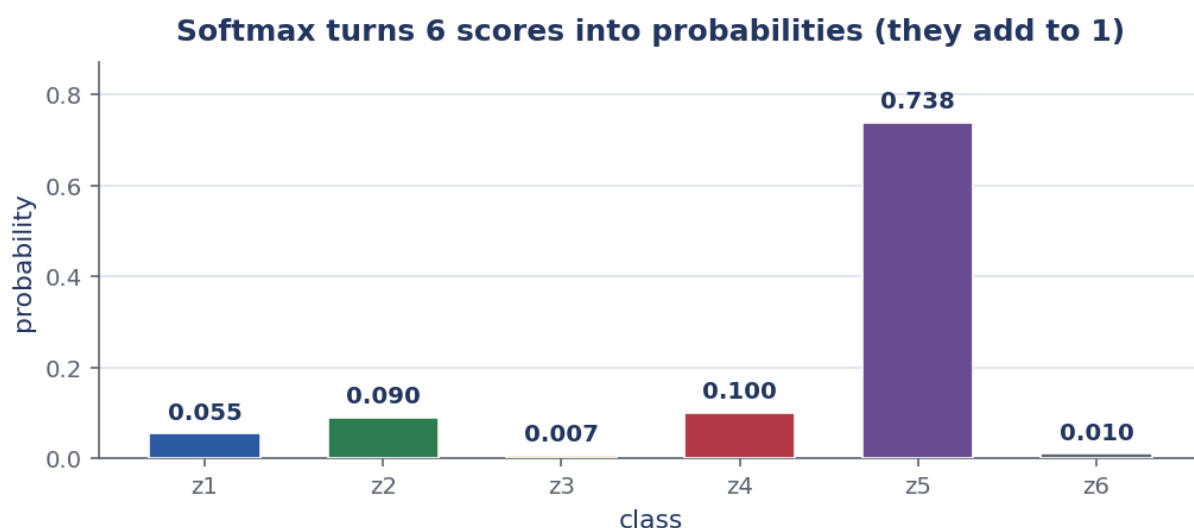


Figure 6: How to read it: each bar is one class’s probability after softmax. The fifth score (3.2) was the largest, so its bar towers over the rest. All six bars add to 1.

a softmax (for many classes). For deeper networks we number the layers with a superscript in square brackets:

$$\mathbf{a}^{[0]} = \mathbf{x}, \quad \mathbf{z}^{[\ell]} = W^{[\ell]} \mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]}, \quad \mathbf{a}^{[\ell]} = g(\mathbf{z}^{[\ell]}). \quad (12)$$

Here  $\ell$  (the Greek letter ell) is the layer number.  $W^{[\ell]}$  is that layer’s weight matrix,  $\mathbf{b}^{[\ell]}$  its bias vector,  $\mathbf{z}^{[\ell]}$  its pre-activation scores, and  $\mathbf{a}^{[\ell]}$  its activated output. The input is  $\mathbf{a}^{[0]} = \mathbf{x}$ . Each layer feeds the next.

## 7.4 Replacing the bias with a dummy input

There is a neat notational trick. Instead of carrying the bias  $b$  separately, add a fake input  $a_0 = 1$  to each layer. Give it a weight  $w_0$ . Then  $w_0 \cdot 1 = w_0$  plays the exact role of the bias. So  $\mathbf{w} \cdot \mathbf{x} + b$  becomes one clean dot product, with the bias folded in as weight  $w_0$  on the constant input 1. Nothing changes in the math; it is just tidier to write.

**Where this shows up in ML.** The softmax is the final layer of almost every classifier with more than two classes. Examples: image labels, the next-token distribution in a language model, and intent labels in a chatbot. The layer notation above is what deep-learning libraries implement, layer by layer.

### ✗ Watch out

Softmax scores are not probabilities until you normalise. The raw scores  $z_i$ , often called **logits**, can be any size, even negative. Only after the exponentials and the division do they become a valid distribution that adds to 1. Skipping the division is a classic bug.

### ✓ Key takeaway

A feedforward network stacks layers, each  $\mathbf{a}^{[\ell]} = g(W^{[\ell]} \mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]})$ . Sigmoid ends a yes/no model; softmax ends a many-class model by turning scores into probabilities that sum to 1.

## 8 Applying networks to NLP tasks

Now we point these networks at language. Two big uses appear in the slides. The first is text classification, like sentiment. The second is representation learning with embeddings, which removes the need to hand-build features.

### 8.1 Text classification and sentiment

#### The intuition

For sentiment, the network reads a review and outputs one number: the chance the review is positive. Simple features can be binary flags, like “does the word great appear?” A plain sigmoid unit on these flags is logistic regression. Add a hidden layer and the network can use feature *combinations*, not just single words.

#### ★ Everyday picture

Judging a movie review is like reading a friend’s text. A single word like “terrible” is a strong clue. But “not terrible” flips the meaning. A flat word-count misses that flip. A hidden layer can learn the combination “not” plus “terrible” and treat it differently.

So a one-layer network handles simple word-flag features. A hidden layer lets the network model nonlinear interactions between features, which is exactly what phrases like “not good” need. For more than two labels, say positive, neutral, and negative, we add more output units and a softmax layer.

#### Sentiment classification: from words to a label



Figure 7: How to read it: a sentiment classifier reads the review, turns words into embeddings, pools them into one fixed-size vector, runs a hidden layer, and ends in a sigmoid or softmax label.

### 8.2 Representation learning with embeddings

Hand-building features is slow and brittle. The modern way is to feed the network **embeddings**: dense vectors that stand for words and are learned from data. Each word  $w$  becomes a short list of numbers  $\mathbf{e}$  that captures its meaning. Similar words get similar vectors.

There is a size problem. Texts vary in length, but a feedforward network wants a fixed-size input. The slides give two fixes.

- **Pad or truncate.** Pick a fixed length. Cut longer texts; pad shorter ones with a blank token. Simple, but it can waste space or lose the tail.
- **Pool into one vector.** Combine all the word vectors into a single **sentence embedding**. Take the element-wise mean (average each dimension) or the element-wise max. One fixed-size vector, any length in.

 **Worked example: pool three 2-D word vectors into one sentence vector**

Say a 3-word text has vectors  $\mathbf{e}_1 = [2, 0]$ ,  $\mathbf{e}_2 = [0, 4]$ ,  $\mathbf{e}_3 = [1, 2]$ . We build one fixed-size vector two ways.

**Step 1. Mean, first dimension.**  $(2 + 0 + 1)/3 = 3/3 = 1$ .

**Step 2. Mean, second dimension.**  $(0 + 4 + 2)/3 = 6/3 = 2$ . So the mean vector is  $[1, 2]$ .

**Step 3. Max, first dimension.**  $\max(2, 0, 1) = 2$ .

**Step 4. Max, second dimension.**  $\max(0, 4, 2) = 4$ . So the max vector is  $[2, 4]$ .

Either way we now have one 2-D vector, no matter that the text had three words. A ten-word text would also collapse to a 2-D vector.

**Where this shows up in ML.** Embeddings power nearly all modern NLP. Word vectors (word2vec, GloVe) were the first wave. Mean-pooling sentence vectors is a strong, cheap baseline for classification. Today's models pool with attention instead, but the goal is the same: turn variable text into fixed-size numbers.

 **Watch out**

Mean-pooling throws away word order. “Dog bites man” and “man bites dog” pool to the same vector. For tasks where order matters, plain averaging is too crude. That limit is one reason recurrent networks and transformers were invented.

 **Key takeaway**

For NLP, feed the network learned embeddings, not hand-built flags. To get a fixed size, pad/truncate, or pool word vectors into one sentence vector by mean or max. A hidden layer captures word combinations.

## 9 Neural language models

A **language model** (LM) predicts the next word, given the words so far. It outputs a probability for every word in the vocabulary. A **neural** language model does this with a neural network. This is the direct ancestor of today’s large language models.

### The intuition

A language model is a very good autocomplete. You type “I made sure the cat gets \_\_\_\_\_” and it ranks every possible next word. “fed” scores high; “purple” scores low. The model’s whole job is that ranking, turned into a probability for each word.

### ★ Everyday picture

Think of texting on your phone. As you type, three word suggestions pop up. The keyboard is guessing your next word from the last few. A neural LM is the same idea, but it guesses over the whole dictionary, not just three options.

### 9.1 Background: chain rule and the Markov shortcut

The probability of a whole sentence factors by the **chain rule**: the chance of each word given all the words before it. For words  $w_1, \dots, w_n$ :

$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1}). \quad (13)$$

Conditioning on *all* prior words is too much. So we make the **Markov assumption**: only the last few words matter. An  $(n - 1)$ -th order model keeps the last  $n - 1$  words. We can back off from a long history down to no history at all:

$$P(w_i \mid w_{i-4}, \dots, w_{i-1}) \rightarrow P(w_i \mid w_{i-2}, w_{i-1}) \rightarrow P(w_i \mid w_{i-1}) \rightarrow P(w_i). \quad (14)$$

Because a real sentence can be any length, we use a fixed-length **sliding window**. We look at a fixed number of prior words, say three, to predict the next, then slide forward by one.

### 9.2 One-hot vectors and the embedding matrix

How does a word enter the network as numbers? First as a **one-hot vector**. This is a list as long as the vocabulary  $|V|$ . It has a 1 at the word’s index and 0 everywhere else. If “toothpaste” is word number 5, its one-hot vector has  $x_5 = 1$  and every other entry 0.

A one-hot is huge and carries no meaning. So an **embedding matrix**  $E$  maps it to a short dense vector. Multiplying  $E$  by a one-hot just selects one column of  $E$ : the word’s embedding. So  $E$  is a lookup table, one dense vector per word.

### 9.3 Why neural LMs beat n-gram LMs

#### The intuition

An n-gram model only knows word *identities*. “cat” and “dog” are just two different symbols to it, as unrelated as “cat” and “Tuesday.” A neural LM knows word *meanings* through embeddings. Since “cat” and “dog” have similar vectors, knowledge about one transfers to the other.

## A fixed 3-word window predicts the next word



Figure 8: How to read it: the three earlier words are the context. The model predicts the highlighted next word  $w_t$ . Slide the window one step right to predict the following word.

Here is the slide’s example. Say the training text has “I have to make sure that the cat gets fed.” But it never has the word “dog” in that spot. Now test on “... make sure that the dog gets \_\_\_\_\_.” An n-gram model has never seen “dog gets,” so it cannot predict “fed.” A neural LM sees that “dog” sits near “cat” in embedding space, and reuses the “cat” pattern to predict “fed.” That sharing is called **generalization**.

### 9.4 The neural LM forward pass

Now the full prediction, step by step. Take the three context words. Look up each one’s embedding in  $E$ . Stick the three embeddings together into one long vector  $\mathbf{e}$  (this is **concatenation**). Pass it through a hidden layer, then to scores, then softmax:

$$\mathbf{e} = [\mathbf{e}_{w_{t-3}}; \mathbf{e}_{w_{t-2}}; \mathbf{e}_{w_{t-1}}], \quad (15)$$

$$\mathbf{h} = g(W\mathbf{e} + \mathbf{b}), \quad (16)$$

$$\mathbf{z} = U\mathbf{h}, \quad (17)$$

$$\mathbf{y} = \text{softmax}(\mathbf{z}). \quad (18)$$

Name each symbol.  $\mathbf{e}$  is the joined context embedding.  $W$  is the hidden weight matrix and  $\mathbf{b}$  its bias.  $g$  is the hidden activation, like ReLU.  $\mathbf{h}$  is the hidden vector.  $U$  is the output weight matrix that produces one score per vocabulary word.  $\mathbf{z}$  is the score vector, of length  $|V|$ . And  $\mathbf{y}$  is the softmax output: a probability for every word in the vocabulary.

### 9.5 Training the embeddings: freeze or learn

Where does  $E$  come from? The slides give two choices.

- **Freeze.** Start  $E$  from a pretraining method, like word2vec. Keep it fixed. Train only  $W$ ,  $U$ , and  $\mathbf{b}$ . Fast, and safe when data is small.
- **Learn jointly.** Let  $E$  update along with the network. Useful when the task needs special representations the generic vectors lack.

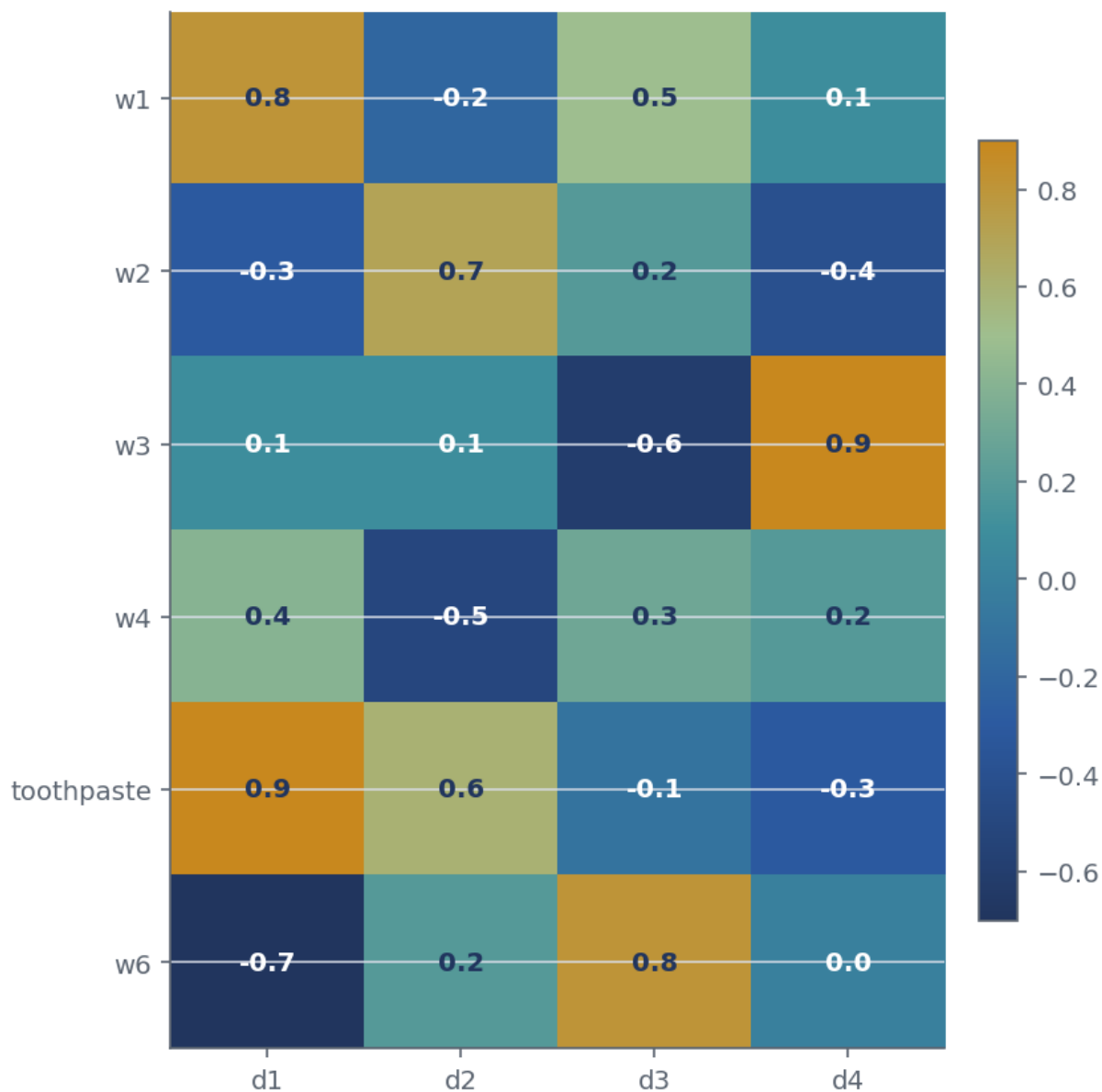
**Embedding matrix E: each row is one word's dense vector**

Figure 9: How to read it: each row of  $E$  is one word's dense vector. Picking “toothpaste” selects its row. The cell colours show the learned numbers; similar words end up with similar rows.

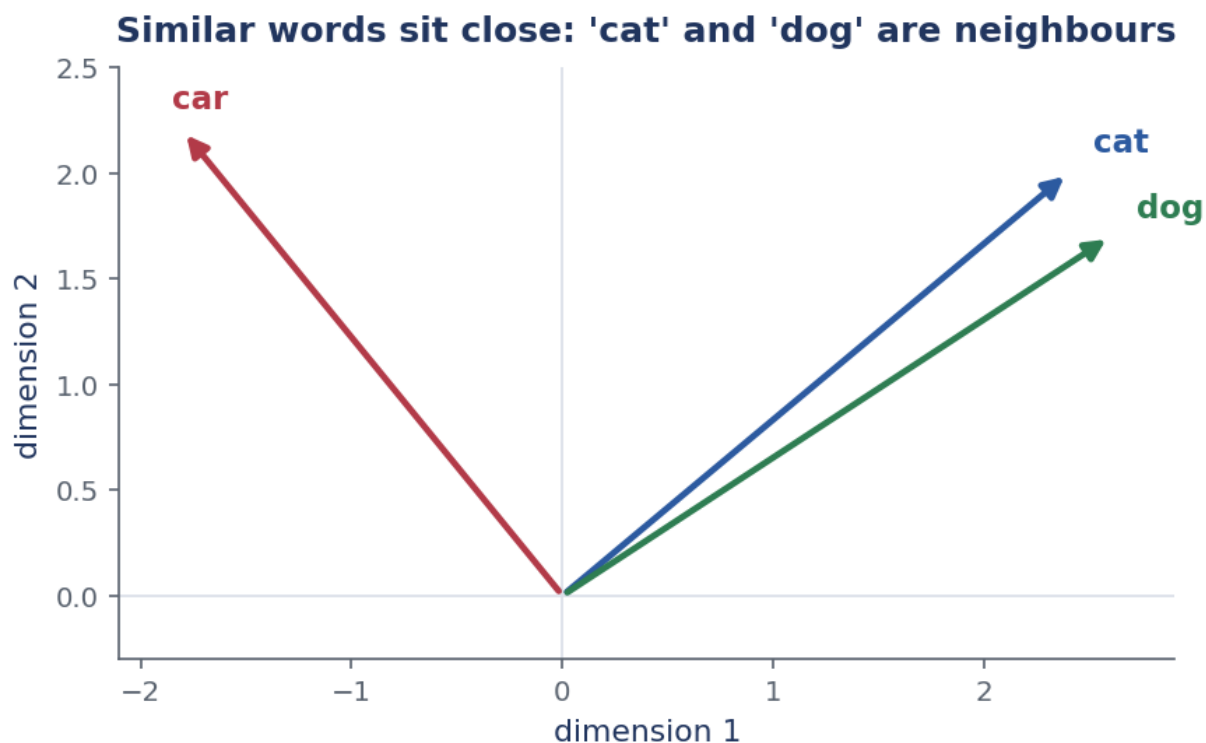


Figure 10: How to read it: each arrow is a word’s embedding. “cat” and “dog” point in almost the same direction, so they are neighbours. “car” points elsewhere. Closeness means similar meaning.

### Neural language model: from words to a next-word distribution

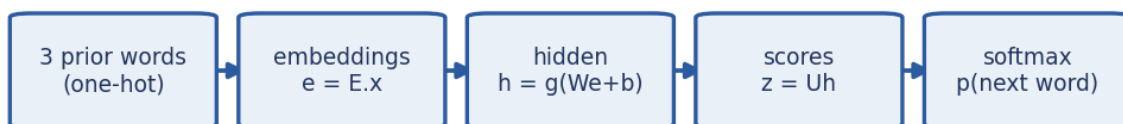


Figure 11: How to read it: three one-hot words become embeddings, which join into  $\mathbf{e}$ . A hidden layer makes  $\mathbf{h}$ , the output layer makes scores  $\mathbf{z}$ , and softmax turns those into a next-word distribution.

## 9.6 N-gram versus neural LM: the trade-off

Neither model wins everywhere. The table below compares them.

| Property         | N-gram LM              | Neural LM                    |
|------------------|------------------------|------------------------------|
| History length   | short (a few words)    | longer contexts              |
| Similar words    | treated as unrelated   | shared via embeddings        |
| Accuracy         | lower on next word     | higher on next word          |
| Speed / cost     | fast, cheap, light     | slower, more data and energy |
| Interpretability | easy to inspect counts | harder to interpret          |
| Best for         | small, simple tasks    | large, rich tasks            |

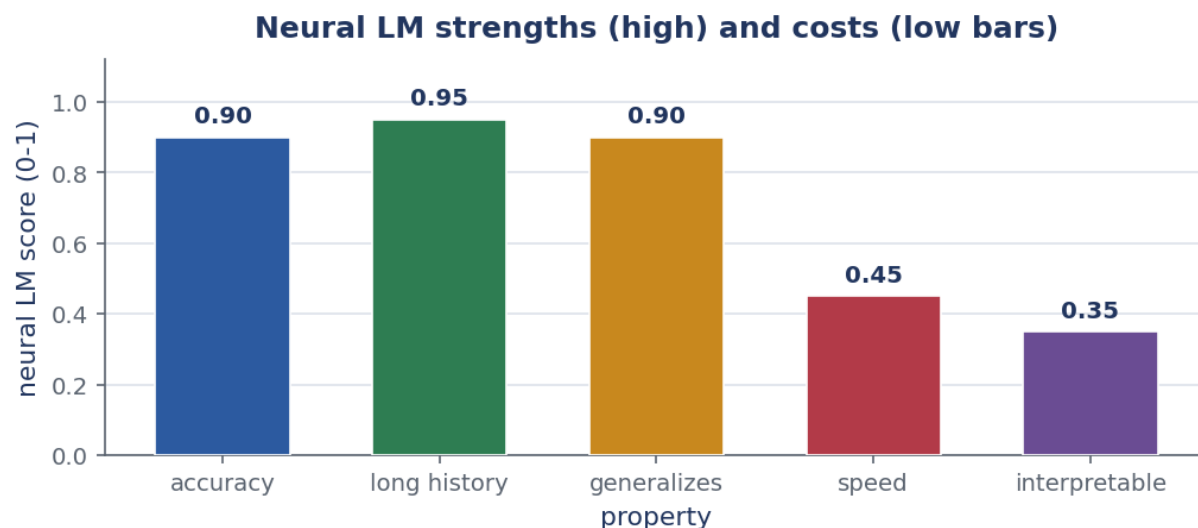


Figure 12: How to read it: tall bars are neural-LM strengths (accuracy, long history, generalizing). Short bars are its costs (slower, harder to interpret). For a small job, the cheaper n-gram can still win.

**Where this shows up in ML.** The neural LM in this section, a feedforward next-word predictor, is the seed Bengio planted in 2003. Swap its fixed window for attention and you get the transformer, the engine of GPT and every modern large language model. The forward pass above is the core idea, scaled up.

### ✗ Watch out

A fixed window is short-sighted. A 3-word window cannot use a clue from ten words back. That hard limit on context is exactly what recurrent networks and, later, transformers were built to fix. Do not expect a small window to track long dependencies.

### ✓ Key takeaway

A neural LM looks up embeddings for the context words, joins them, runs a hidden layer, and softmaxes to a next-word distribution. Because embeddings encode meaning, it generalizes across similar words, which n-gram models cannot.



## 10 Training neural networks

A network starts with random weights and predicts badly. Training fixes that. We measure how wrong each prediction is with a **loss**, then nudge every weight to make the loss smaller. We repeat this many times. Full backpropagation is next lecture; here we build the core idea on one unit.

### The intuition

Training is downhill walking. The loss is a landscape: high where the model is wrong, low where it is right. The slope of the loss tells us which way is uphill. We step the opposite way, downhill, by a small amount. Repeat until we reach a low valley.

### ★ Everyday picture

Picture hiking down a hill in thick fog. You cannot see the valley. But you can feel which way the ground slopes under your boots. So you take a small step downhill, feel again, and step again. Each step lowers you a little.

### 10.1 The cross-entropy loss

For a yes/no model we use the **cross-entropy loss**. Let  $y$  be the true label (0 or 1) and  $\hat{y}$  the model's predicted probability of class 1. The loss for one example is:

$$L_{\text{CE}} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]. \quad (19)$$

Read it as surprise. If the true label is 1, only the first term survives, and the loss is  $-\log \hat{y}$ . A confident-and-right guess ( $\hat{y}$  near 1) gives near-zero loss. A confident-and-wrong guess ( $\hat{y}$  near 0) blows the loss up. So the loss punishes confident mistakes hard.

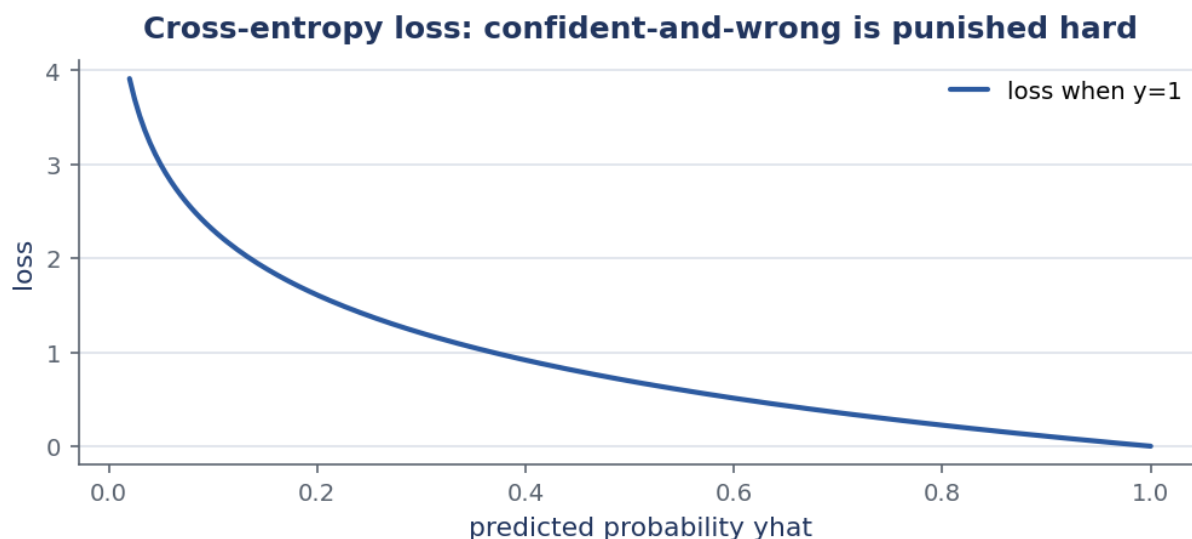


Figure 13: How to read it: this is the loss when the true label is 1. As the predicted probability rises toward 1, the loss falls to 0. As it falls toward 0, the loss shoots up. Be confident only when right.

### 10.2 Gradient descent: one downhill step

To shrink the loss, we adjust each weight  $w$  against its slope. The update rule is:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}. \quad (20)$$

Here  $\eta$  (the Greek letter eta) is the **learning rate**: how big a step we take. The term  $\partial L / \partial w$  is the slope of the loss with respect to that weight. The minus sign turns “uphill slope” into a “downhill step.” To find the slope we use the **chain rule**. For a function built as  $f(x) = u(v(x))$ , the slope is a product of slopes:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}. \quad (21)$$

For deeper networks, we apply this chain rule through a **computation graph**, backward from the loss to each weight. That backward sweep is **backpropagation**, the topic of the next lecture.

### 10.3 The logistic-regression gradient, derived

For one sigmoid unit something beautiful happens. The messy chain rule collapses into a clean result. Let us derive it. The unit computes  $z = \mathbf{w} \cdot \mathbf{x} + b$ , then  $\hat{y} = \sigma(z)$ , with the cross-entropy loss above.

 **Worked example: derive  $\partial L / \partial w_j = (\hat{y} - y) x_j$ , then take one step**

We chain three small slopes, then plug in numbers for one step.

**Step 1. Slope of loss in  $\hat{y}$ .**  $\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$ .

**Step 2. Slope of sigmoid in  $z$ .** A known fact:  $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$ .

**Step 3. Multiply the first two.**  $\frac{\partial L}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \hat{y}(1-\hat{y})$ . Expanding gives  $-y(1-\hat{y}) + (1-y)\hat{y} = -y + y\hat{y} + \hat{y} - y\hat{y} = \hat{y} - y$ .

**Step 4. Slope of  $z$  in  $w_j$ .** Since  $z = \sum_k w_k x_k + b$ , we get  $\frac{\partial z}{\partial w_j} = x_j$ .

**Step 5. Chain them.**  $\frac{\partial L}{\partial w_j} = (\hat{y} - y) x_j$ . The clean result: prediction error times the input.

**Step 6. One numeric step.** Take  $\hat{y} = 0.7$ , true  $y = 1$ ,  $x_j = 2$ , learning rate  $\eta = 0.1$ . The error is  $\hat{y} - y = 0.7 - 1 = -0.3$ . The gradient is  $(-0.3)(2) = -0.6$ . The update is  $w_j \leftarrow w_j - 0.1 \times (-0.6) = w_j + \mathbf{0.06}$ .

**Step 7. Sense-check.** The label was 1 but we guessed 0.7, too low. The step raised  $w_j$ , which (for a positive input) pushes the next guess higher. The fix points the right way.

So for logistic regression the gradient is simply  $(\hat{y} - y) x_j$ : how wrong we were, scaled by the input. No messy terms remain.

**Where this shows up in ML.** This loop, loss then gradient then step, is how every neural network is trained, from a tiny classifier to GPT. Real models have millions of weights, and backpropagation computes the slope for all of them at once. But each weight takes the exact step above.

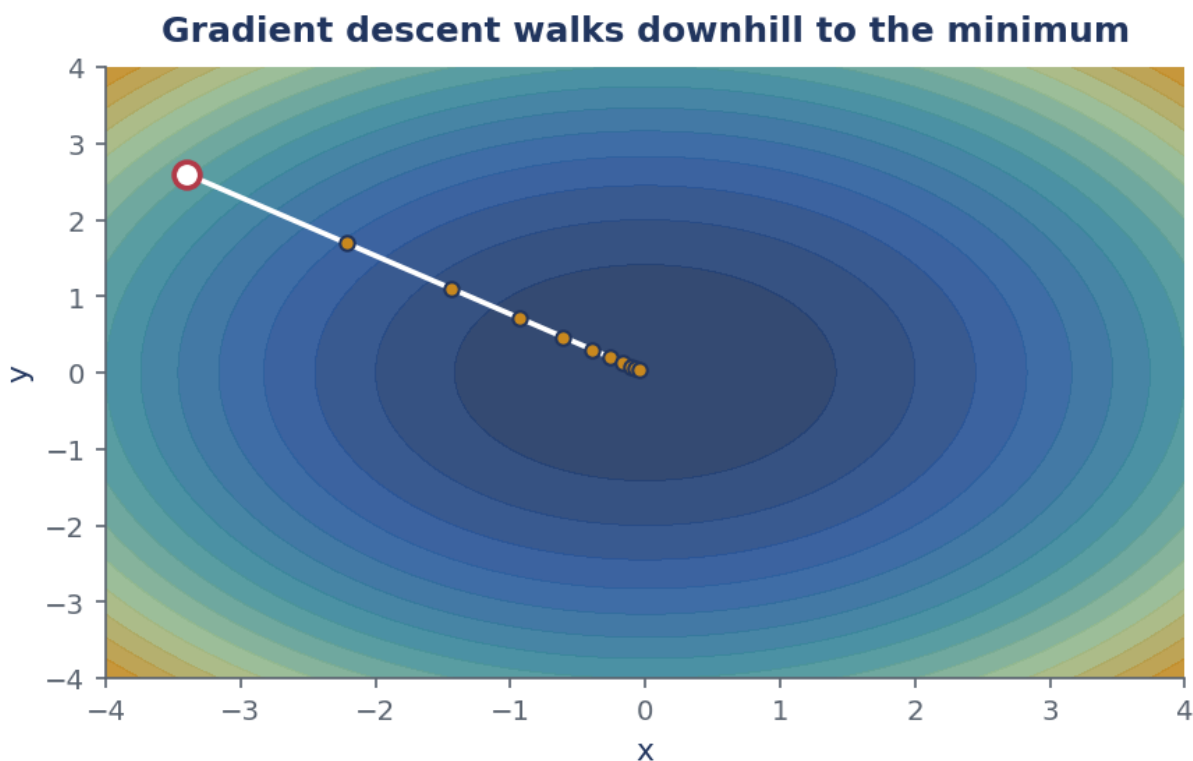


Figure 14: How to read it: the colours are a bowl-shaped loss, dark at the bottom (low loss). The dotted path is a sequence of downhill steps. Each step moves toward the centre, the minimum.

#### ⚠ Watch out

The learning rate  $\eta$  is delicate. Too small and training crawls for ages. Too large and the steps overshoot the valley, bouncing or even diverging to infinity. Picking  $\eta$  well is one of the most important practical skills in deep learning.

#### ✓ Key takeaway

Train by gradient descent:  $w \leftarrow w - \eta \partial L / \partial w$ , with the cross-entropy loss. For one sigmoid unit the gradient simplifies to  $(\hat{y} - y) x_j$ , error times input. Backpropagation extends this to deep networks.

## 11 Large language models and generative AI

A **large language model** (LLM) is just a very large neural language model. It is usually a transformer, trained on a huge amount of text, to predict the next **token** (a word or word-piece). Scale changes everything. At billions of parameters, an LLM can write, summarize, translate, and answer questions.

### The intuition

An LLM is the next-word predictor from earlier, grown enormous. Same core job: given the text so far, guess what comes next. But with vast data and size, that single skill turns into something that looks like broad language ability.

### ★ Everyday picture

Think of a student who has read almost everything. Ask a question, and they continue your sentence with a sensible, well-read answer. They are not looking facts up live. They are predicting what a knowledgeable person would say next.

**Benefits and uses.** Because one model handles many tasks, LLMs power chat assistants, drafting and editing, code help, summaries, translation, and search. One trained model, many jobs, with no task-specific retraining for each.

### 11.1 Prompts, completions, and prompt engineering

You talk to an LLM with a **prompt**: the text you give it. It replies with a **completion**: the text it generates. **Prompt engineering** is the craft of writing prompts that get good completions. Small wording changes can change the output a lot.

### 11.2 In-context learning: zero-shot and few-shot

LLMs can learn a task from the prompt alone, with no weight changes. This is **in-context learning**. Two styles appear in the slides.

- **Zero-shot.** You describe the task and give zero examples. The model relies on what it already learned.
- **Few-shot.** You include a few worked examples in the prompt. The model copies the pattern.

### Worked example: a sentiment prompt, zero-shot versus few-shot

We ask the model to label a review as Positive or Negative.

**Step 1. Zero-shot prompt.** We write: “Classify the sentiment as Positive or Negative. Review: The plot dragged and I was bored. Sentiment:”. No examples are given.

**Step 2. Zero-shot result.** The model completes with **Negative**, using its general knowledge of the words.

**Step 3. Few-shot prompt.** We add two examples first: “Review: I loved every minute. Sentiment: Positive. Review: A total waste of time. Sentiment: Negative. Review: The plot dragged and I was bored. Sentiment:”.

**Step 4. Few-shot result.** The model again answers **Negative**, now also matching the exact label format we showed (one word, capitalized).

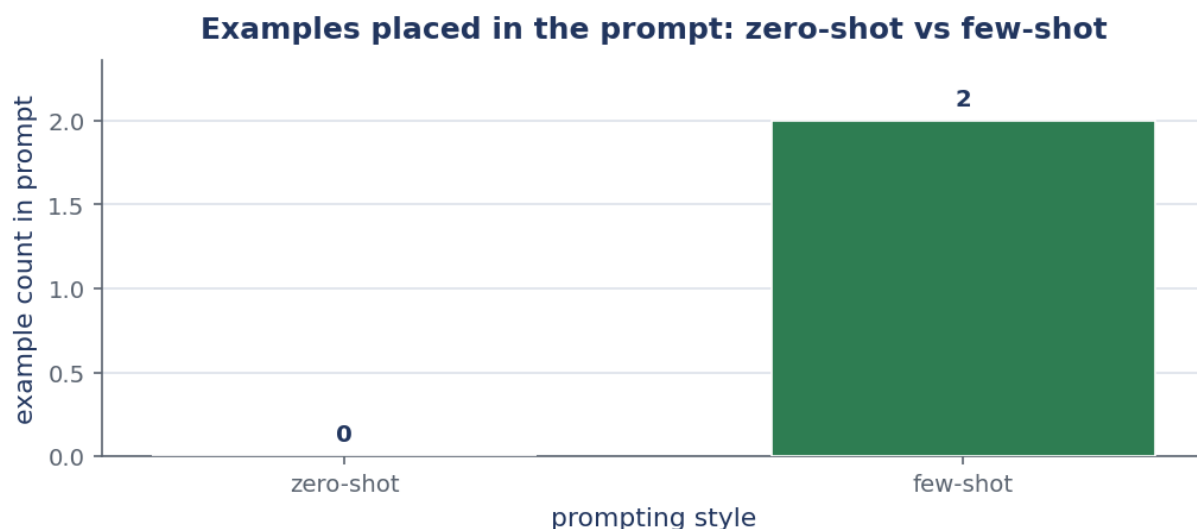


Figure 15: How to read it: the bars count examples placed inside the prompt. Zero-shot gives none; few-shot gives a handful. More examples often steer the model better, at the cost of a longer prompt.

Both work here. Few-shot helps most when the task is unusual or the output format must be exact.

### 11.3 Generative AI, assistants, and tools

**Generative AI** means models that create new content: text, images, audio, or code. LLMs are the text branch. The slides list common shapes:

- **AI assistants**, in several flavours. A notification assistant alerts you. An FAQ assistant answers common questions. A contextual one uses your situation. A personalized one adapts to you. An autonomous one acts on its own.
- **Generative tools**, such as image generation (text to picture) and code generation (text to working code).

**Where this shows up in ML.** An LLM is the scaled-up cousin of the neural LM you built by hand. Prompt engineering is now a core skill for using these models. Zero-shot and few-shot prompting let one frozen model do new tasks with no retraining, which is a large practical win.

#### ⚠ Watch out

LLMs predict plausible text, not verified truth. They can state wrong facts with full confidence; this is called **hallucination**. Always check important outputs. A fluent answer is not the same as a correct answer.

#### ✓ Key takeaway

An LLM is a giant neural LM, usually a transformer, trained to predict the next token. You steer it with prompts. Zero-shot gives no examples; few-shot gives a few. Both adapt the model with no weight changes.

## 12 Real-life challenges and transfer learning

Deep learning is hungry for labelled data, often more than 50,000 examples. Two problems follow. The source data (what the model trained on) and the target data (what you will use it on) should match in style. And labelled target data is often scarce. **Transfer learning** solves both.

### The intuition

Transfer learning is reusing what a model already learned. Instead of starting from scratch, you take a model trained on a big task and adapt it to your smaller one. The early layers already know useful, general patterns; you only retrain the last part.

### ★ Everyday picture

If you can already drive a car, learning to drive a truck is fast. You keep most of the skill (steering, signals, road sense) and adjust only a few things (size, mirrors). You do not relearn driving from zero. That reuse is transfer learning.

The usual recipe: take a pre-trained model, **freeze** its learned weights (hold them fixed), and add new layers for your task. You train only the new layers on your small target dataset. The slides name two kinds of transfer.

- **Transductive transfer.** No labelled target data. You adapt a model from one domain to another. This is **domain adaptation**.
- **Inductive transfer.** You do have labelled target data. This covers multi-task learning and pretraining, like learning word embeddings first, then using them.

**Fine-tuning** goes one step further. You let the pre-trained layers update too, not just the new ones. This works best when your new dataset is large and similar to the original. With little data, fine-tuning everything can overfit, so freezing is safer.

### Transfer learning: reuse what a big model already learned



Figure 16: How to read it: a model is pre-trained on big source data. You freeze its learned layers, add new task layers, and train those on small target data. Most knowledge is reused, not rebuilt.

### Worked example: cars to trucks, what to freeze and what to train

You have a strong car-recognizer and a small truck dataset. Plan the transfer.

**Step 1. Start from the pre-trained model.** The car model's early layers already detect edges, wheels, and shapes.

**Step 2. Freeze the early layers.** Hold those general feature detectors fixed. They

work for trucks too.

**Step 3. Replace the final layer.** Swap the car-specific output for a new truck output layer, with fresh random weights.

**Step 4. Train only the new layer.** Fit it on the small truck dataset. Few weights to learn means little data is enough.

**Step 5. Optional fine-tune.** If the truck dataset grows large, unfreeze and gently update the earlier layers too, for a small extra gain.

You reused most of the network and trained only the part that had to change.

**Where this shows up in ML.** Transfer learning is everywhere: image and speech recognition, named-entity recognition, sentiment, cross-lingual tasks, even game-playing. Modern NLP runs on it. You take a pre-trained language model and fine-tune it on your task, instead of training from scratch.

**✗ Watch out**

Transfer helps only when source and target are related. A model pre-trained on photos will not transfer well to medical scans without care. And fine-tuning all layers on tiny data can overfit fast. Match the domains, and freeze when data is scarce.

**✓ Key takeaway**

Deep learning needs lots of labelled data, which is often scarce. Transfer learning reuses a pre-trained model: freeze its layers, add and train new ones. Fine-tuning updates the old layers too, best when target data is large and similar.

## Glossary & symbols

---

### Key terms.

|                            |                                                                       |
|----------------------------|-----------------------------------------------------------------------|
| <b>unit (neuron)</b>       | the atom of a network: a weighted sum, a bias, then an activation.    |
| <b>weight</b>              | a number that says how much one input matters.                        |
| <b>bias</b>                | one extra number added to the sum; it shifts the decision.            |
| <b>activation</b>          | the nonlinear bend applied to the weighted sum.                       |
| <b>sigmoid</b>             | squashes any number into the range $(0, 1)$ ; reads as a probability. |
| <b>tanh</b>                | an S-shape centred on zero, range $(-1, 1)$ .                         |
| <b>ReLU</b>                | keeps positives, sets negatives to zero; the common default.          |
| <b>perceptron</b>          | the simplest unit; a hard 0/1 output with no smooth activation.       |
| <b>linearly separable</b>  | two classes that one straight line can split.                         |
| <b>XOR</b>                 | fires when the two inputs differ; not linearly separable.             |
| <b>hidden layer</b>        | a middle layer that builds useful features from the input.            |
| <b>feedforward network</b> | units in layers, data flowing one way; an MLP.                        |
| <b>softmax</b>             | turns a list of scores into probabilities that add to 1.              |
| <b>logits</b>              | the raw scores before softmax; can be any size.                       |
| <b>embedding</b>           | a word's dense vector; similar words sit close.                       |
| <b>one-hot vector</b>      | a vector with a single 1 at a word's index, 0 elsewhere.              |
| <b>embedding matrix</b>    | a lookup table; one dense vector per word.                            |
| <b>language model</b>      | predicts the next word as a probability over the vocabulary.          |
| <b>Markov assumption</b>   | only the last few words matter for the next word.                     |
| <b>sliding window</b>      | a fixed-length context that moves one word at a time.                 |
| <b>generalization</b>      | reusing knowledge across similar inputs (cat to dog).                 |
| <b>cross-entropy</b>       | the loss for classification; punishes confident mistakes.             |
| <b>gradient descent</b>    | step weights against the slope to shrink the loss.                    |
| <b>learning rate</b>       | how big each downhill step is; written $\eta$ .                       |
| <b>backpropagation</b>     | the chain rule run backward to get every weight's slope.              |
| <b>LLM</b>                 | a very large neural language model, usually a transformer.            |



|                          |                                                        |
|--------------------------|--------------------------------------------------------|
| <b>prompt</b>            | the text you give an LLM; the completion is its reply. |
| <b>zero-shot</b>         | a prompt with no examples; few-shot has a few.         |
| <b>transfer learning</b> | reuse a pre-trained model on a new task.               |
| <b>fine-tuning</b>       | also update the pre-trained layers, not just new ones. |

### Symbols at a glance.

| Symbol                                                 | Meaning                                                 |
|--------------------------------------------------------|---------------------------------------------------------|
| $x_i$                                                  | one input value (a feature) to a unit                   |
| $w_i$                                                  | the weight on input $x_i$                               |
| $b$                                                    | the bias: a single number added to the weighted sum     |
| $z$                                                    | the weighted sum, $z = \mathbf{w} \cdot \mathbf{x} + b$ |
| $\mathbf{w} \cdot \mathbf{x}$                          | the dot product: multiply matching parts and add        |
| $f, g$                                                 | an activation function (the nonlinear bend)             |
| $\sigma(z)$                                            | the sigmoid, $1/(1 + e^{-z})$ ; maps to $(0, 1)$        |
| $\text{ReLU}(z)$                                       | $\max(0, z)$ : keep positives, zero negatives           |
| $y, \hat{y}$                                           | the true label, and the model's predicted probability   |
| $\mathbf{h}$                                           | the hidden-layer output vector                          |
| $W, U$                                                 | weight matrices (hidden layer, output layer)            |
| $W^{[\ell]}, \mathbf{a}^{[\ell]}, \mathbf{z}^{[\ell]}$ | layer $\ell$ 's weights, activated output, and scores   |
| $\text{softmax}(\mathbf{z})_i$                         | $e^{z_i} / \sum_j e^{z_j}$ : score $i$ as a probability |
| $ V $                                                  | the vocabulary size (number of words)                   |
| $E$                                                    | the embedding matrix; one dense vector per word         |
| $\mathbf{e}$                                           | a word embedding, or the joined context embedding       |
| $L_{\text{CE}}$                                        | the cross-entropy loss                                  |
| $\eta$                                                 | the learning rate (Greek letter eta); step size         |
| $\partial L / \partial w$                              | the slope of the loss with respect to weight $w$        |

**Further reading.** Jurafsky & Martin, *Speech and Language Processing* (3rd ed.), Chapter 7: Neural Networks and Neural Language Models.